

Manual Tecnico

Sistema de Gestion Empresarial

Proyecto: App Web Empresarial

Tipo de sistema: Aplicacion web full-stack

Base de datos: PostgreSQL mediante Prisma ORM

1. Introduccion

El presente manual tecnico describe el funcionamiento interno del sistema **App Web Empresarial**, una aplicacion web orientada a la gestion de usuarios, clientes, proyectos, tareas, roles y permisos. El sistema esta construido bajo una arquitectura cliente-servidor, donde el frontend se encarga de la interfaz visual y el backend administra la logica de negocio, seguridad, validaciones y comunicacion con la base de datos.

Como estudiante de ingenieria, puedo interpretar este proyecto como una solucion empresarial modular, ya que separa responsabilidades en diferentes capas: presentacion, servicios HTTP, controladores, servicios de negocio, validaciones, seguridad y persistencia de datos. Esta separacion facilita el mantenimiento, la escalabilidad y la comprension del codigo.

El proyecto esta organizado como un monorepo, es decir, contiene en una misma carpeta tanto la aplicacion cliente como el servidor.

2. Objetivo del sistema

El objetivo principal del sistema es administrar informacion empresarial mediante una plataforma web centralizada. Sus funciones principales son:

- Gestionar usuarios del sistema.
- Gestionar clientes.
- Crear y administrar proyectos.
- Crear y asignar tareas a usuarios.
- Controlar roles y permisos.
- Autenticar usuarios mediante JWT.
- Registrar cambios importantes en una bitacora de auditoria.

3. Alcance del sistema

El sistema permite que un usuario autenticado pueda ingresar a una interfaz web y, segun los permisos asignados a su rol, realizar operaciones sobre los modulos disponibles. La aplicacion esta pensada para un entorno empresarial donde existen proyectos asociados a clientes y tareas asignadas a responsables.

Los modulos principales son:

- Autenticacion.
- Usuarios.
- Roles.

- Clientes.
- Proyectos.
- Tareas.
- Auditoria.
- Dashboard o pagina principal.

4. Tecnologias utilizadas

4.1 Frontend

El frontend esta ubicado en la carpeta cliente/.

Tecnologia	Uso dentro del sistema
Angular	Framework principal para construir la interfaz web.
Angular Router	Manejo de rutas y navegacion entre pantallas.
Angular Forms	Manejo de formularios de login, usuarios, clientes, proyectos, tareas y roles.
Angular HttpClient	Comunicacion HTTP con el backend.
Angular Signals	Manejo reactivo del estado de sesion.
Angular Material	Componentes visuales y elementos de interfaz.
Tailwind CSS	Estilos utilitarios para el diseno visual.
RxJS	Manejo de observables y respuestas asincronas.
jsPDF y jspdf-autotable	Generacion de reportes PDF desde el frontend.

4.2 Backend

El backend esta ubicado en la carpeta servidor/.

Tecnologia	Uso dentro del sistema
NestJS	Framework backend basado en Node.js y TypeScript.
Prisma ORM	Comunicacion con la base de datos.
PostgreSQL	Motor de base de datos configurado en Prisma.
JWT	Autenticacion mediante access token y refresh token.
Passport JWT	Estrategias para validar tokens JWT.

bcryptjs	Cifrado de contraseñas y refresh tokens.
Zod	Validacion de datos recibidos en peticiones HTTP.
Jest	Pruebas unitarias y e2e.

5. Arquitectura general

Arquitectura general del sistema

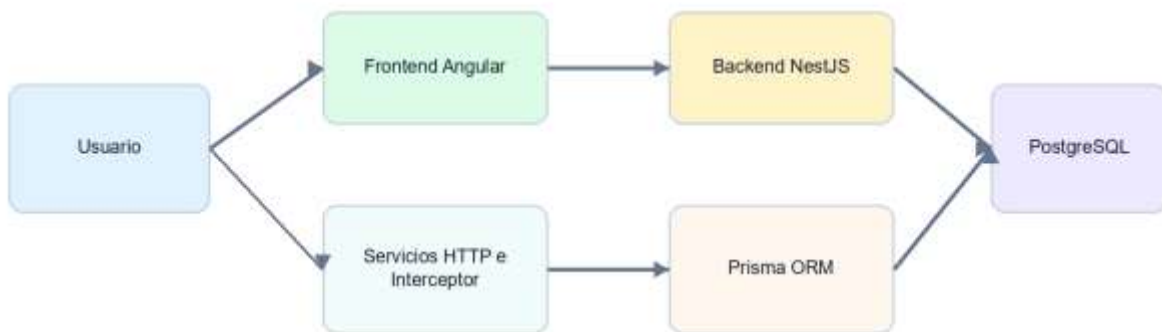


Figura 1. Arquitectura general cliente-servidor del sistema.

La arquitectura del sistema es de tipo cliente-servidor:

```

Usuario
|
v
Frontend Angular
|
| Peticiones HTTP con token JWT
v
Backend NestJS
|
| Prisma ORM
v
Base de datos PostgreSQL
  
```

El flujo basico es el siguiente:

1. El usuario ingresa sus credenciales en la pantalla de login.
2. El frontend envia las credenciales al backend.
3. El backend valida el usuario y genera tokens JWT.

4. El frontend guarda los tokens en localStorage.
5. En cada petición protegida, el frontend envía el access token en el header Authorization.
6. El backend valida el token y revisa los permisos del usuario.
7. Si el usuario tiene permiso, se ejecuta la operación solicitada.
8. Prisma realiza la consulta o modificación en la base de datos.
9. El backend devuelve una respuesta JSON al frontend.

6. Estructura del proyecto

La estructura principal del proyecto es:

```
app-web-empresarial-main/  
  cliente/  
    src/  
      app/  
        core/  
          guards/  
          interceptors/  
          services/  
        clients/  
        dashboard/  
        layout/  
        login/  
        pages/  
        projects/  
        roles/  
        shared/  
        tasks/  
        users/  
      environments/  
      angular.json  
      package.json  
  servidor/  
    prisma/  
      schema.prisma  
      seed.ts  
      migrations/  
    src/  
      auth/  
      audit-log/  
      clients/  
      common/  
      prisma/  
      projects/  
      roles/  
      tasks/  
      users/  
    package.json  
  README.md
```

```
package.json
pnpm-workspace.yaml
```

Esta division permite trabajar frontend y backend de forma independiente, pero manteniendolos dentro del mismo repositorio.

7. Configuracion del proyecto

7.1 Scripts generales

En el package.json raiz existen scripts para ejecutar las partes del sistema:

```
{
  "servidor": "pnpm --filter servidor dev",
  "cliente": "pnpm --filter cliente dev",
  "dev": "pnpm -r dev"
}
```

Esto indica que el proyecto usa pnpm como gestor en el monorepo, aunque el cliente tambien contiene package-lock.json, por lo que puede instalarse con npm dentro de cliente/.

7.2 Variables de entorno del backend

El backend necesita variables de entorno para conectarse a la base de datos y firmar tokens:

```
DATABASE_URL="postgresql://usuario:password@host:puerto/base_de_datos"
JWT_SECRET="clave_para_access_token"
JWT_REFRESH_SECRET="clave_para_refresh_token"
PORT=3000
```

7.3 Variables de entorno del frontend

El frontend maneja la URL de la API desde los archivos de ambiente:

```
// environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000',
}
// environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://app-web-empresarial.onrender.com',
}
```

8. Backend

El backend esta desarrollado con NestJS. NestJS promueve una estructura modular basada en modulos, controladores y servicios.

8.1 Archivo principal

El archivo principal del backend es:

```
servidor/src/main.ts
```

Este archivo crea la aplicacion NestJS y activa CORS:

```
app.enableCors({
  origin: '*',
  methods: 'GET,HEAD,PUT,PATCH,POST,DELETE,OPTIONS',
  allowedHeaders: 'Content-Type, Authorization, X-Requested-With, Accept, Origin',
  credentials: true
});
```

Tambien levanta el servidor en el puerto definido por PORT o en 3000 por defecto.

8.2 Modulo principal

El modulo principal es:

servidor/src/app.module.ts

Este modulo importa:

- PrismaModule
- AuthModule
- UsersModule
- ClientsModule
- ProjectsModule
- TasksModule
- RolesModule
- AuditLogModule

Cada modulo representa una funcionalidad del sistema.

9. Modulos del backend

9.1 AuthModule

El modulo de autenticacion se encarga del inicio de sesion, cierre de sesion y renovacion de tokens.

Controlador:

servidor/src/auth/auth.controller.ts

Servicio:

servidor/src/auth/auth.service.ts

Endpoints:

Metodo	Ruta	Descripcion
POST	/auth/login	Valida credenciales y devuelve usuario, access token y refresh token.
POST	/auth/logout	Cierra sesion e invalida el refresh token guardado.
POST	/auth/refresh	Genera nuevos tokens usando un refresh token valido.

El login realiza estos pasos:

10. Busca al usuario por correo.
11. Incluye el rol y permisos asociados.
12. Compara la contraseña enviada con la contraseña cifrada usando bcrypt.
13. Genera un access token y un refresh token.
14. Guarda el refresh token cifrado en la base de datos.
15. Devuelve los datos del usuario y sus tokens.

El access token tiene duracion de 15 minutos y el refresh token tiene duracion de 7 dias.

9.2 UsersModule

Gestiona los usuarios del sistema.

Controlador:

`servidor/src/users/users.controller.ts`

Servicio:

`servidor/src/users/users.service.ts`

Endpoints:

Metodo	Ruta	Descripcion
GET	/users	Lista todos los usuarios.
GET	/users/roles	Lista los roles disponibles para asignar a usuarios.
GET	/users/:id	Obtiene un usuario por ID.
POST	/users	Crea un usuario.
PATCH	/users/:id	Actualiza un usuario.
DELETE	/users/:id	Elimina un usuario.

Validaciones principales:

- Email valido.
- Nombre minimo de 3 caracteres.
- Contraseña minima de 6 caracteres.
- roleId valido en formato CUID.

Cuando se crea o actualiza una contraseña, se cifra con bcrypt.

9.3 ClientsModule

Gestiona clientes empresariales.

Controlador:

`servidor/src/clients/clients.controller.ts`

Servicio:

`servidor/src/clients/clients.service.ts`

Endpoints:

Metodo	Ruta	Descripcion
GET	/clients	Lista todos los clientes.
GET	/clients/:id	Obtiene un cliente por ID.
POST	/clients	Crea un cliente.
PATCH	/clients/:id	Actualiza un cliente.
DELETE	/clients/:id	Elimina un cliente.

Validaciones principales:

- Nombre minimo de 2 caracteres.
- Email valido y unico.
- Telefono opcional, numerico y de 8 digitos.
- Estado permitido: activo o inactivo.

9.4 ProjectsModule

Gestiona proyectos asociados a clientes.

Controlador:

`servidor/src/projects/projects.controller.ts`

Servicio:

`servidor/src/projects/projects.service.ts`

Endpoints:

Metodo	Ruta	Descripcion
GET	/projects	Lista proyectos.
GET	/projects/:id	Obtiene un proyecto por ID.
POST	/projects	Crea un proyecto.
PATCH	/projects/:id	Actualiza un proyecto.
DELETE	/projects/:id	Elimina un proyecto.

Campos principales:

- name
- description
- startDate
- endDate
- status
- clientId

Estados permitidos:

- pendiente
- en_progreso
- completado
- cancelado

Una validacion importante es que la fecha de finalizacion no puede ser anterior a la fecha de inicio.

Cuando se crea, actualiza o elimina un proyecto, el sistema genera un registro en la bitacora de auditoria.

9.5 TasksModule

Gestiona tareas asociadas a proyectos y asignadas a usuarios.

Controlador:

servidor/src/tasks/tasks.controller.ts

Servicio:

servidor/src/tasks/tasks.service.ts

Endpoints:

Metodo	Ruta	Descripcion
GET	/tasks	Lista tareas.
GET	/tasks?projectId=id	Lista tareas filtradas por proyecto.
GET	/tasks/:id	Obtiene una tarea por ID.
POST	/tasks	Crea una tarea.
PATCH	/tasks/:id	Actualiza una tarea.
DELETE	/tasks/:id	Elimina una tarea.

Prioridades permitidas:

- baja
- media
- alta

Estados permitidos:

- pendiente
- en_progreso
- completado
- bloqueado

Cada tarea pertenece a un proyecto y tiene un usuario responsable.

9.6 RolesModule

Gestiona roles y permisos.

Controlador:

servidor/src/roles/roles.controller.ts

Servicio:

servidor/src/roles/roles.service.ts

Endpoints:

Metodo	Ruta	Descripcion
GET	/roles	Lista roles con sus permisos.
GET	/roles/permissions	Lista todos los permisos disponibles.
GET	/roles/:id	Obtiene un rol por ID.
POST	/roles	Crea un rol.
PATCH	/roles/:id	Actualiza un rol.
DELETE	/roles/:id	Elimina un rol.

El sistema impide eliminar el rol ADMIN, lo cual es correcto porque evita dejar el sistema sin un rol administrativo principal.

9.7 AuditLogModule

Gestiona la bitacora de auditoria.

Controlador:

servidor/src/audit-log/audit-log.controller.ts

Servicio:

servidor/src/audit-log/audit-log.service.ts

Endpoint:

Metodo	Ruta	Descripcion
GET	/audit-log	Lista registros de auditoria.
GET	/audit-log?projectId=id	Filtra auditoria por proyecto.
GET	/audit-log?entityId=id	Filtra auditoria por entidad.

La auditoria registra:

- Tipo de entidad.
- ID de la entidad.
- Nombre de la entidad.
- Accion realizada.
- Datos anteriores.

- Datos nuevos.
- Usuario que realizo la accion.
- Fecha de creacion.

10. Seguridad del backend

Flujo de autenticación JWT

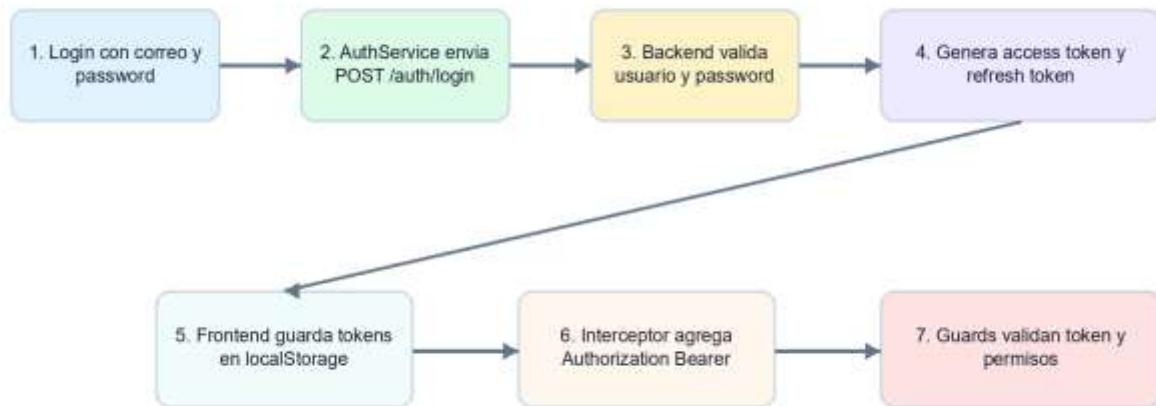


Figura 2. Flujo de autenticacion y autorizacion mediante JWT.

10.1 Autenticacion JWT

El sistema utiliza dos tokens:

- **Access token:** se usa para acceder a rutas protegidas.
- **Refresh token:** se usa para renovar la sesion.

El access token se envia en cada peticion de esta forma:

Authorization: Bearer <access_token>

10.2 Guards

Los guards son clases que deciden si una peticion puede continuar o no.

Guard	Funcion
AuthGuard	Verifica que exista un access token valido.
RtGuard	Verifica el refresh token.
PermissionsGuard	Verifica que el usuario tenga los permisos requeridos.

10.3 Decorador de permisos

El decorador `@Permissions()` permite declarar que permisos necesita una ruta.

Ejemplo:

```
@Permissions(PermissionSlug.CREAR_USUARIOS)
@Post()
async create(@Body() dto: CreateUserDto) {
  return this.usersService.create(dto);
}
```

Esto significa que solo un usuario con permiso `CREAR_USUARIOS` puede crear usuarios.

10.4 Cifrado de contraseñas

Las contraseñas no se guardan en texto plano. Antes de guardar un usuario, la contraseña se cifra usando `bcrypt.hash()`.

En el login, la comparación se hace con:

```
bcrypt.compare(dto.password, user.password)
```

Esto mejora la seguridad porque aunque alguien acceda a la base de datos, no obtiene las contraseñas originales.

11. Base de datos

Modelo conceptual de base de datos

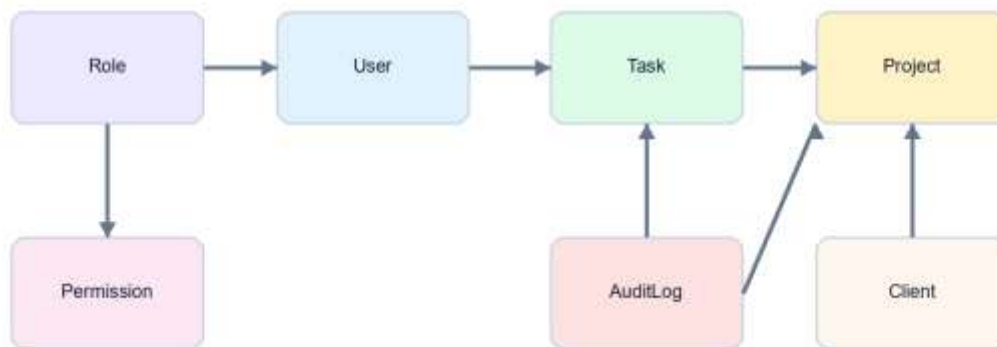


Figura 3. Modelo conceptual de entidades y relaciones principales.

La base de datos se define en:

```
servidor/prisma/schema.prisma
```

El proveedor configurado es PostgreSQL:

```
datasource db {
  provider = "postgresql"
```

```
url    = env("DATABASE_URL")
}
```

11.1 Modelos principales

User

Representa a los usuarios del sistema.

Campos importantes:

- id
- email
- name
- password
- refreshToken
- roleId
- createdAt
- updatedAt

Relaciones:

- Un usuario pertenece a un rol.
- Un usuario puede tener muchas tareas asignadas.

Role

Representa un rol de usuario.

Campos:

- id
- name
- description

Relaciones:

- Un rol puede tener muchos usuarios.
- Un rol puede tener muchos permisos.

Permission

Representa un permiso del sistema.

Campos:

- id
- slug
- name

Relacion:

- Un permiso puede pertenecer a muchos roles.

Client

Representa a los clientes.

Campos:

- id

- name
- email
- phone
- company
- status
- createdAt
- updatedAt

Relacion:

- Un cliente puede tener muchos proyectos.

Project

Representa proyectos de clientes.

Campos:

- id
- name
- description
- startDate
- endDate
- status
- clientId
- createdAt
- updatedAt

Relaciones:

- Un proyecto pertenece a un cliente.
- Un proyecto puede tener muchas tareas.

Task

Representa tareas dentro de un proyecto.

Campos:

- id
- title
- description
- priority
- status
- startDate
- endDate
- projectId
- userId
- createdAt
- updatedAt

Relaciones:

- Una tarea pertenece a un proyecto.
- Una tarea se asigna a un usuario.

AuditLog

Representa la bitacora de cambios.

Campos:

- id
- entityType
- entityId
- entityName
- action
- previousData
- newData
- userId
- userName
- projectId
- createdAt

12. Relaciones de la base de datos

Role 1 --- N User

Role N --- N Permission

Client 1 --- N Project

Project 1 --- N Task

User 1 --- N Task

Project 1 --- N AuditLog

Explicación:

- Un rol puede estar asignado a varios usuarios.
- Un rol puede tener muchos permisos y un permiso puede estar en varios roles.
- Un cliente puede tener varios proyectos.
- Un proyecto puede tener varias tareas.
- Un usuario puede tener varias tareas asignadas.
- La bitacora puede guardar cambios relacionados con proyectos y tareas.

13. Seed de la base de datos

El archivo de seed se encuentra en:

servidor/prisma/seed.ts

Este archivo crea:

- Permisos iniciales.
- Rol ADMIN.
- Rol USUARIO.
- Usuario administrador inicial.

Credenciales iniciales:

Correo: admin@gmail.com

Contraseña: 12345678

El rol ADMIN recibe todos los permisos. El rol USUARIO recibe permisos limitados, principalmente para ver usuarios, ver proyectos asignados, ver tareas y actualizar tareas.

14. Frontend

Relacion entre modulos del frontend y backend

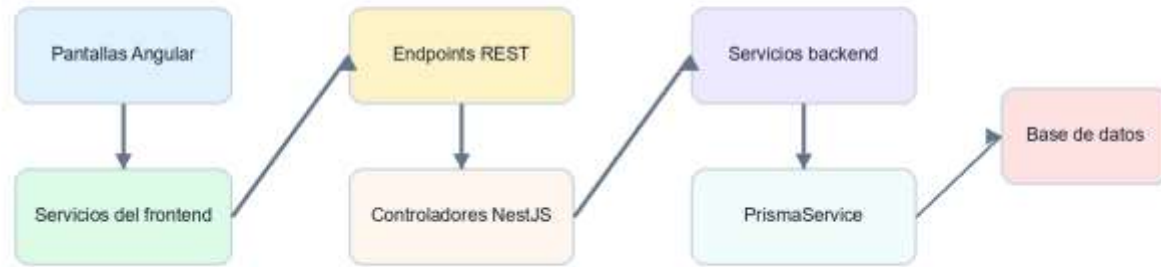


Figura 4. Relacion entre modulos visuales, servicios HTTP y backend.

El frontend esta desarrollado con Angular y se ubica en:

cliente/

Angular funciona mediante componentes. Cada pantalla o bloque visual se implementa como un componente.

14.1 Configuración principal

El archivo:

cliente/src/app/app.config.ts

configura:

- Router.
- Animaciones.
- Cliente HTTP.
- Interceptores.

Los interceptores configurados son:

- authInterceptor
- timeoutInterceptor

14.2 Rutas del frontend

Las rutas estan definidas en:

cliente/src/app/app.routes.ts

Rutas principales:

Ruta	Componente	Protegida
------	------------	-----------

/login	LoginComponent	No
/home	HomeComponent	Si
/users	UsersComponent	Si
/clients	ClientsComponent	Si
/projects	ProjectsComponent	Si
/tasks	TasksComponent	Si
/roles	RolesComponent	Si

La ruta raiz usa LayoutComponent, que funciona como estructura general de la aplicacion autenticada.

14.3 AuthService

Archivo:

cliente/src/app/core/services/auth.service.ts

Responsabilidades:

- Iniciar sesion.
- Guardar usuario y tokens.
- Restaurar la sesion desde localStorage.
- Cerrar sesion.
- Consultar permisos del usuario.

El estado del usuario se maneja con signal:

```
private _user = signal<any>(null);
user = computed(() => this._user());
isLoggedIn = computed(() => !!this._user());
permissions = computed(() => this._user()?.permissions || []);
```

Esto permite que la interfaz reaccione cuando cambia el estado de autenticacion.

14.4 AuthGuard

Archivo:

cliente/src/app/core/guards/auth.guard.ts

Este guard evita que un usuario no autenticado entre a las rutas protegidas. Si no hay sesion, redirige a /login.

14.5 AuthInterceptor

Archivo:

cliente/src/app/core/interceptors/auth.interceptor.ts

Este interceptor agrega el token a las peticiones:

Authorization: Bearer <token>

Tambien maneja errores:

- Si recibe 401, considera que la sesion expiro.

- Si recibe 403, considera que no hay permisos suficientes.
- Limpia el almacenamiento local y redirige al login.

14.6 Servicios HTTP del frontend

Cada modulo tiene un servicio que encapsula las peticiones HTTP al backend.

Servicio	Endpoint base
UserService	/users
ClientsService	/clients
ProjectsService	/projects
TasksService	/tasks
RolesService	/roles
AuditLogService	/audit-log

Esto es una buena practica porque evita que los componentes llamen directamente al HttpClient para todo. En lugar de eso, los componentes consumen metodos como findAll, create, update o remove.

15. Componentes principales del frontend

15.1 LoginComponent

Permite ingresar al sistema mediante correo y contrasena. Usa AuthService.login() para enviar las credenciales al backend.

15.2 LayoutComponent

Funciona como plantilla principal despues del login. Normalmente contiene menu, navegacion y el espacio donde se renderizan las rutas hijas.

15.3 HomeComponent / Dashboard

Es la pagina principal del sistema despues de iniciar sesion. Sirve como punto de entrada para visualizar informacion general.

15.4 UsersComponent y UserFormComponent

Permiten listar, crear, editar y eliminar usuarios. El formulario trabaja con datos como:

- Nombre.
- Correo.
- Contraseña.
- Rol asignado.

15.5 ClientsComponent y ClientFormComponent

Permiten administrar clientes. Los datos principales son:

- Nombre.

- Correo.
- Telefono.
- Empresa.
- Estado.

15.6 ProjectsComponent y ProjectFormComponent

Permiten administrar proyectos. Los proyectos estan relacionados con clientes y contienen fechas, estado y descripcion.

Tambien existen componentes adicionales:

- project-gantt: representa una vista tipo cronograma.
- project-report: genera reportes del proyecto.
- project-history: muestra historial o auditoria relacionada.

15.7 TasksComponent y TaskFormComponent

Permiten administrar tareas. Cada tarea esta asociada a:

- Un proyecto.
- Un usuario responsable.
- Una prioridad.
- Un estado.
- Fechas de inicio y fin.

15.8 RolesComponent y RoleFormComponent

Permiten administrar roles y permisos. Desde este modulo se puede definir que acciones puede realizar cada tipo de usuario.

15.9 Componentes compartidos

En shared/components/ existen componentes reutilizables:

- button
- input
- modal

Esto ayuda a mantener consistencia visual y reduce repeticion de codigo.

16. Flujo de autenticacion completo

1. Usuario ingresa correo y contrasena.
2. LoginComponent llama a AuthService.login().
3. AuthService envia POST /auth/login.
4. AuthService recibe usuario, access_token y refresh_token.
5. El frontend guarda datos en localStorage.
6. El usuario entra a rutas protegidas.
7. AuthInterceptor agrega el access token a cada peticion.
8. AtGuard valida el token en el backend.
9. PermissionsGuard valida permisos cuando la ruta los requiere.
10. Si todo es correcto, se ejecuta el controlador.

17. Control de acceso RBAC

RBAC significa **Role-Based Access Control**, o control de acceso basado en roles.

En este sistema:

- Cada usuario tiene un rol.
- Cada rol tiene permisos.
- Cada endpoint protegido exige ciertos permisos.
- Si el usuario no tiene el permiso requerido, el backend responde con error 403.

Ejemplo:

Usuario -> Rol ADMIN -> Permisos completos

Usuario -> Rol USUARIO -> Permisos limitados

Permisos definidos:

VER_USUARIOS
CREAR_USUARIOS
ACTUALIZAR_USUARIOS
ELIMINAR_USUARIOS
VER_CLIENTES
CREAR_CLIENTES
ACTUALIZAR_CLIENTES
ELIMINAR_CLIENTES
VER_PROYECTOS
VER_PROYECTOS_ASIGNADOS
CREAR_PROYECTOS
ACTUALIZAR_PROYECTOS
ELIMINAR_PROYECTOS
VER_TAREAS
CREAR_TAREAS
ACTUALIZAR_TAREAS
ELIMINAR_TAREAS

18. Validacion de datos

El backend usa Zod para validar datos antes de procesarlos.

Ejemplo en usuarios:

email: z.string().email('Email invalido')

password: z.string().min(6, 'La contraseña debe tener al menos 6 caracteres')

Esto evita que lleguen datos incorrectos a la base de datos y mejora la estabilidad del sistema.

El pipe encargado de ejecutar la validacion es:

servidor/src/common/pipes/zod.pipe.ts

19. Instalacion y ejecucion local

19.1 Requisitos previos

Se recomienda tener instalado:

- Node.js.
- npm.
- pnpm.
- PostgreSQL o una base de datos compatible.

19.2 Instalacion del backend

```
cd servidor  
pnpm install
```

Crear archivo .env:

```
DATABASE_URL="postgresql://usuario:password@localhost:5432/app_web_empresarial"  
JWT_SECRET="clave_access"  
JWT_REFRESH_SECRET="clave_refresh"  
PORT=3000
```

Ejecutar migraciones:

```
npx prisma migrate dev
```

Ejecutar seed:

```
npx prisma db seed
```

Iniciar servidor:

```
pnpm dev
```

19.3 Instalacion del frontend

```
cd cliente  
npm install  
npm run dev
```

La aplicacion Angular normalmente queda disponible en:

```
http://localhost:4200
```

El backend queda disponible en:

```
http://localhost:3000
```

20. Despliegue

Segun el README del proyecto, el despliegue de produccion usa:

Parte	Plataforma
Frontend	Vercel
Backend	Render
Base de datos	Neon PostgreSQL

URLs registradas:

Frontend: <https://app-web-empresarial.vercel.app/login>

Backend: <https://app-web-empresarial.onrender.com>

Para produccion, el frontend toma la API desde `environment.prod.ts`.

21. Pruebas

21.1 Backend

El backend tiene configuracion de Jest.

Comandos:

```
pnpm test
pnpm test:e2e
pnpm test:cov
```

21.2 Frontend

El frontend tiene comando de pruebas Angular:

```
npm test
```

22. Buenas practicas identificadas

En el proyecto se observan buenas practicas como:

- Separacion entre frontend y backend.
- Uso de arquitectura modular en NestJS.
- Uso de servicios para separar la logica de negocio.
- Uso de DTOs y validaciones con Zod.
- Uso de Prisma como capa de acceso a datos.
- Uso de JWT para autenticacion.
- Uso de bcrypt para cifrar contraseñas.
- Uso de guards para proteger rutas.
- Uso de interceptores en Angular para adjuntar tokens.
- Uso de componentes compartidos en frontend.
- Uso de auditoria para registrar cambios importantes.

23. Observaciones tecnicas y posibles mejoras

Como parte del analisis tecnico, se identifican algunas mejoras posibles:

16. En ProjectsController, la validacion manual de permisos compara contra textos como ver:proyectos, mientras que los permisos reales del sistema usan constantes como VER_PROYECTOS. Se recomienda unificar esa comparacion usando PermissionSlug.VER_PROYECTOS y PermissionSlug.VER_PROYECTOS_ASIGNADOS.
17. El AuthInterceptor del frontend actualmente redirige al login cuando recibe 401, pero no intenta usar el refresh token para renovar la sesion. Como el backend ya tiene /auth/refresh, se podria implementar renovacion automatica.
18. El backend permite CORS con origin: '*'. Para produccion, es mas seguro limitarlo al dominio real del frontend.
19. El servicio de auditoria captura errores pero no los reporta. Para mantenimiento, seria mejor registrar el error en consola o en un sistema de logs.
20. Se recomienda documentar ejemplos de peticiones con cuerpos JSON para facilitar pruebas en Postman o Insomnia.
21. Se recomienda evitar guardar informacion sensible innecesaria en localStorage, ya que puede ser vulnerable ante ataques XSS. Una alternativa seria usar cookies HTTP-only.

24. Ejemplos de peticiones

24.1 Login

POST /auth/login

Content-Type: application/json

```
{  
  "email": "admin@gmail.com",  
  "password": "12345678"  
}
```

24.2 Crear cliente

POST /clients

Authorization: Bearer <access_token>

Content-Type: application/json

```
{  
  "name": "Cliente Demo",  
  "email": "cliente@demo.com",  
  "phone": "12345678",  
  "company": "Empresa Demo",  
  "status": "activo"  
}
```

24.3 Crear proyecto

POST /projects

Authorization: Bearer <access_token>

Content-Type: application/json

```
{  
  "name": "Proyecto Demo",  
  "description": "Proyecto de prueba para documentacion",  
  "startDate": "2026-05-11",  
  "endDate": "2026-06-11",  
  "status": "pendiente",  
  "clientId": "id_del_cliente"  
}
```

24.4 Crear tarea

POST /tasks

Authorization: Bearer <access_token>

Content-Type: application/json

```
{  
  "title": "Análisis de requerimientos",  
  "description": "Revisar las necesidades del cliente",  
  "priority": "media",  
  "status": "pendiente",  
  "projectId": "id_del_proyecto",  
  "userId": "id_del_usuario",  
  "startDate": "2026-05-11",  
  "endDate": "2026-05-15"  
}
```

25. Conclusion

El sistema App Web Empresarial es una aplicacion full-stack bien estructurada para administrar informacion empresarial. Su arquitectura separa correctamente la interfaz del usuario, la logica de negocio y la persistencia de datos. El backend con NestJS ofrece una organizacion clara por modulos, mientras que Prisma facilita el trabajo con la base de datos.

Desde el punto de vista de ingenieria, el sistema demuestra conceptos importantes como autenticacion JWT, control de acceso basado en roles, validacion de datos, consumo de APIs REST, diseno modular y auditoria de cambios. Estos elementos son fundamentales en aplicaciones empresariales reales, ya que ayudan a construir software mantenible, seguro y escalable.